COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	September 17, 2026	**DATE SUBMITTED:**	September 21, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.
Task 1:

generalTree.h

```cpp
#ifndef GENERALTREE_H
#define GENERALTREE_H

template<typename T>
class BinarySearchTree {
protected:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    //helper findData
    bool searchHelper(int choice, T targetKey) {
        if (choice == 1) { //Preorder: Root -> Left -> Right
            if (this->key == targetKey) return true;
            if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
            if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
        }
        else if (choice == 2) { //Inorder: Left -> Root -> Right
            if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
            if (this->key == targetKey) return true;
            if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
        }
        else if (choice == 3) { // Postorder: Left -> Right -> Root
            if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
            if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
            if (this->key == targetKey) return true;
        }
        return false;
    }

```

```
31  public:
32      //Constructor
33      BinarySearchTree(T value);
34
35      //BST Insertion
36      void insert(T value);
37
38      //for Binary Tree and General Tree linking
39      void setLeft(BinarySearchTree<T>* node);
40      void setRight(BinarySearchTree<T>* node);
41      BinarySearchTree<T>* getLeft() const;
42      BinarySearchTree<T>* getRight() const;
43      T getKey() const;
44
45      //for General Tree LeftChild / RightSibling
46      void addChild(BinarySearchTree<T>* child);
47
48      //traversal:
49      void preOrder();
50      void inOrder();
51      void postOrder();
52
53      //findData function
54      void findData(int choice, T targetKey);
55  };
56
57  template<typename T>
58  BinarySearchTree<T>::BinarySearchTree(T value) {
```

```
59      key = value;
60      left = nullptr;
61      right = nullptr;
62  }
63
64  template<typename T>
65  void BinarySearchTree<T>::insert(T value) {
66      if (value < key) {
67          if (left == nullptr) {
68              left = new BinarySearchTree(value);
69          } else {
70              left->insert(value);
71          }
72      }
73      else if (value > key) {
74          if (right == nullptr) {
75              right = new BinarySearchTree(value);
76          } else {
77              right->insert(value);
78          }
79      }
80  }
81
82  template<typename T>
83  void BinarySearchTree<T>::setLeft(BinarySearchTree<T>* node) {
84      left = node;
85  }
86
87  template<typename T>
```

```cpp
void BinarySearchTree<T>::setRight(BinarySearchTree<T>* node) {
    right = node;
}

template<typename T>
BinarySearchTree<T>* BinarySearchTree<T>::getLeft() const {
    return left;
}

template<typename T>
BinarySearchTree<T>* BinarySearchTree<T>::getRight() const {
    return right;
}

template<typename T>
T BinarySearchTree<T>::getKey() const {
    return key;
}

//attach a child node using Left-Child (child) / Right-Sibling (sibling)
template<typename T>
void BinarySearchTree<T>::addChild(BinarySearchTree<T>* child) {
    if (!child) return;

    if (this->left == nullptr) {
        this->left = child;
    } else {
        BinarySearchTree<T>* sibling = this->left;
        while (sibling->right != nullptr) {
            sibling = sibling->right;
```

```
118             }
119             sibling->right = child;
120         }
121     }
122
123     template<typename T>
124     void BinarySearchTree<T>::preOrder() {
125         std::cout << key << " ";
126         if (left != nullptr) {
127             left->preOrder();
128         }
129         if (right != nullptr) {
130             right->preOrder();
131         }
132     }
133
134     template<typename T>
135     void BinarySearchTree<T>::inOrder() {
136         if (left != nullptr) {
137             left->inOrder();
138         }
139         std::cout << key << " ";
140         if (right != nullptr) {
141             right->inOrder();
142         }
143     }
144
```

```cpp
template<typename T>
void BinarySearchTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";
}

//find data
template<typename T>
void BinarySearchTree<T>::findData(int choice, T targetKey) {
    if (searchHelper(choice, targetKey)) {
        std::cout << targetKey << " was found!" << std::endl;
    }
}

```

main.cpp

```cpp
#include <iostream>
#include "generalTree.h"

int main() {
    //creating nodes
    BinarySearchTree<char>* A = new BinarySearchTree<char>('A');
    BinarySearchTree<char>* B = new BinarySearchTree<char>('B');
    BinarySearchTree<char>* C = new BinarySearchTree<char>('C');
    BinarySearchTree<char>* D = new BinarySearchTree<char>('D');
    BinarySearchTree<char>* E = new BinarySearchTree<char>('E');
    BinarySearchTree<char>* F = new BinarySearchTree<char>('F');
    BinarySearchTree<char>* G = new BinarySearchTree<char>('G');
    BinarySearchTree<char>* H = new BinarySearchTree<char>('H');
    BinarySearchTree<char>* I = new BinarySearchTree<char>('I');
    BinarySearchTree<char>* J = new BinarySearchTree<char>('J');
    BinarySearchTree<char>* K = new BinarySearchTree<char>('K');
    BinarySearchTree<char>* L = new BinarySearchTree<char>('L');
    BinarySearchTree<char>* M = new BinarySearchTree<char>('M');
    BinarySearchTree<char>* N = new BinarySearchTree<char>('N');
    BinarySearchTree<char>* P = new BinarySearchTree<char>('P');
    BinarySearchTree<char>* Q = new BinarySearchTree<char>('Q');

    // Link root children
    A->addChild(B);
    A->addChild(C);
    A->addChild(D);
    A->addChild(E);
    A->addChild(F);
    A->addChild(G);

```

```
31      // Link Level 1 children
32      D->addChild(H);
33      E->addChild(I);
34      E->addChild(J);
35      F->addChild(K);
36      F->addChild(L);
37      F->addChild(M);
38      G->addChild(N);
39
40      // Link Level 2 children
41      J->addChild(P);
42      J->addChild(Q);
43
44      std::cout << "Tree successfully created" << std::endl;
45
46      std::cout << "Pre-order:  ";
47      A->preOrder();
48      std::cout << std::endl;
49
50      std::cout << "Post-order: ";
51      A->postOrder();
52      std::cout << std::endl;
53
54      std::cout << "In-order:   ";
55      A->inOrder();
56      std::cout << std::endl;
57
58      std::cout << "\ntesting findData" << std::endl;
59      A->findData(1, 'J'); //J was found!
60      A->findData(1, 'Z'); //No output
61
```

output

```
Tree successfully created
Pre-order:  A B C D H E I J P Q F K L M G N
Post-order: H Q P J I M L K N G F E D C B A
In-order:   B C H D I P Q J E K L M F N G A

testing findData
J was found!

adding O
O was found!
```

Task 2:

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2

P	0	3
Q	0	3

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and postorder traversal.

Task 3:

3.1

Pre-order	A → B → C → D → H → E → I → J → P → Q → F → K → L → M → G → N
Post-order	B → C → H → D → I → P → Q → J → E → K → L → M → F → N → G → A
In-order	B → A → C → H → D → I → E → P → J → Q → K → F → L → M → N → G

3.2

Pre-order traversal

generalTree.h

```
123  template<typename T>
124  void BinarySearchTree<T>::preOrder() {
125      std::cout << key << " ";
126      if (left != nullptr) {
127          left->preOrder();
128      }
129      if (right != nullptr) {
130          right->preOrder();
131      }
132  }
```

main.cpp

```
46      std::cout << "Pre-order:  ";
47      A->preOrder();
48      std::cout << std::endl;
```

output

```
Pre-order:  A B C D H E I J P Q F K L M G N
```

observation

My answer from 3.1 is correct because pre-order traversal follows the root → left → right pattern.

In-order traversal

generalTree.h

```
134  template<typename T>
135  void BinarySearchTree<T>::inOrder() {
136      if (left != nullptr) {
137          left->inOrder();
138      }
139      std::cout << key << " ";
140      if (right != nullptr) {
141          right->inOrder();
142      }
143  }
```

main.cpp

```
54      std::cout << "In-order:    ";
55      A->inOrder();
56      std::cout << std::endl;
```

output

```
In-order:    B C H D I P Q J E K L M F N G A
```

observation

My answer from 3.1 is correct because in-order traversal follows the left→ right → root pattern.

Post=order traversal

generalTree.h

```
145   template<typename T>
146   void BinarySearchTree<T>::postOrder() {
147       if (left != nullptr) {
148           left->postOrder();
149       }
150       if (right != nullptr) {
151           right->postOrder();
152       }
153       std::cout << key << " ";
154   }
```

main.cpp

```
50      std::cout << "Post-order: ";
51      A->postOrder();
52      std::cout << std::endl;
```

output

```
Post-order: H Q P J I M L K N G F E D C B A
```

observation

My answer from 3.1 is correct because post-order traversal follows the left → root → right pattern.

3.3

Find Data

generalTree.h

```
11      //helper findData
12      bool searchHelper(int choice, T targetKey) {
13          if (choice == 1) { //Preorder: Root -> Left -> Right
14              if (this->key == targetKey) return true;
15              if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
16              if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
17          }
18          else if (choice == 2) { //Inorder: Left -> Root -> Right
19              if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
20              if (this->key == targetKey) return true;
21              if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
22          }
23          else if (choice == 3) { // Postorder: Left -> Right -> Root
24              if (left != nullptr && left->searchHelper(choice, targetKey)) return true;
25              if (right != nullptr && right->searchHelper(choice, targetKey)) return true;
26              if (this->key == targetKey) return true;
27          }
28          return false;
29      }
```

```
53      //findData function
54      void findData(int choice, T targetKey);
```

```
156  //find data
157  template<typename T>
158  void BinarySearchTree<T>::findData(int choice, T targetKey) {
159      if (searchHelper(choice, targetKey)) {
160          std::cout << targetKey << " was found!" << std::endl;
161      }
162  }
```

main.cpp

```
58      std::cout << "\ntesting findData" << std::endl;
59      A->findData(1, 'J'); //J was found!
60      A->findData(1, 'Z'); //No output
```

output

```
testing findData
J was found!
```

3.4

main.cpp

```
62      std::cout << "\nadding O" << std::endl;
63      BinarySearchTree<char>* O = new BinarySearchTree<char>('O');
64      G->addChild(O);
65      A->findData(1, 'O'); // O was found!
```

output

```
adding O
O was found!
```

answer

The letter 'O' was successfully located in the tree after being inserted as a child node of 'G'.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

Screenshot of Code

BST.h

```
#ifndef BST_H
#define BST_H

template<typename T>
class BinarySearchTree{
    private:
        T key;
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;
    public:
    //constructor
    BinarySearchTree(T value);
    // insertion:
    void insert(T value);

    //traversal:
    void preOrder();
    void inOrder();
    void postOrder();
};

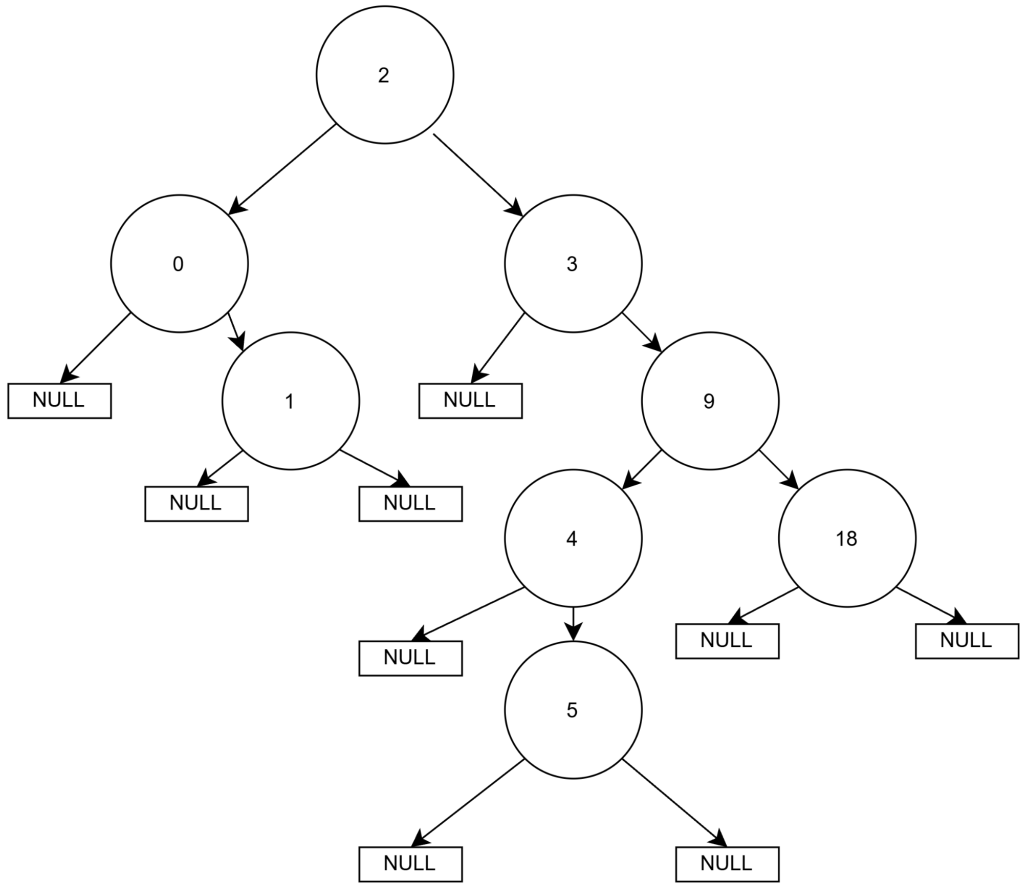
template<typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
```

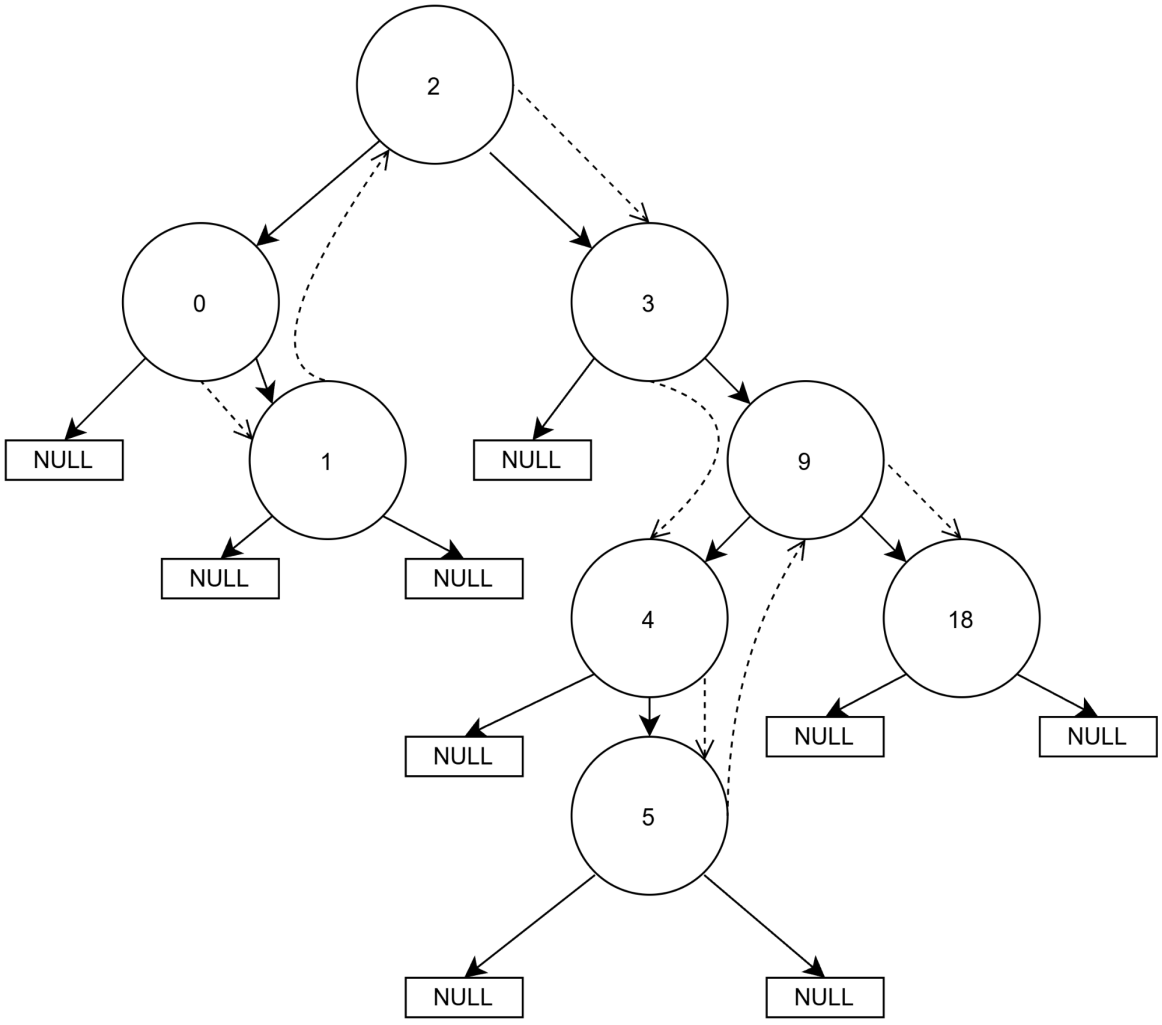
```
26      right = nullptr;
27  }
28
29  template<typename T>
30  void BinarySearchTree<T>::insert(T value){
31      // go to left if: value is < key
32      if(value < key){
33
34          if(left == nullptr){
35              left = new BinarySearchTree(value);
36          } else{
37              left->insert(value);
38          }
39      }
40      else if(value > key){
41          if (right == nullptr){
42              right = new BinarySearchTree<T>(value);
43          } else{
44              right->insert(value);
45          }
46      }
47      // go to right if: value > key
48  }
```

```
49
50    template<typename T>
51  void BinarySearchTree<T>::preOrder(){
52        std::cout << key << " ";
53        if(left != nullptr){
54            left->preOrder();
55        }
56        if (right != nullptr){
57            right->preOrder();
58        }
59    }
60
61    template<typename T>
62  void BinarySearchTree<T>::inOrder(){
63        if(left != nullptr){
64            left->inOrder();
65        }
66        std::cout << key << " ";
67
68        if (right != nullptr){
69            right->inOrder();
70        }
71    }
72
```

```
73  template<typename T>
74  void BinarySearchTree<T>::postOrder(){
75      if(left != nullptr){
76          left->postOrder();
77      }
78
79      if (right != nullptr){
80          right->postOrder();
81      }
82      std::cout << key << " ";
83  }
```

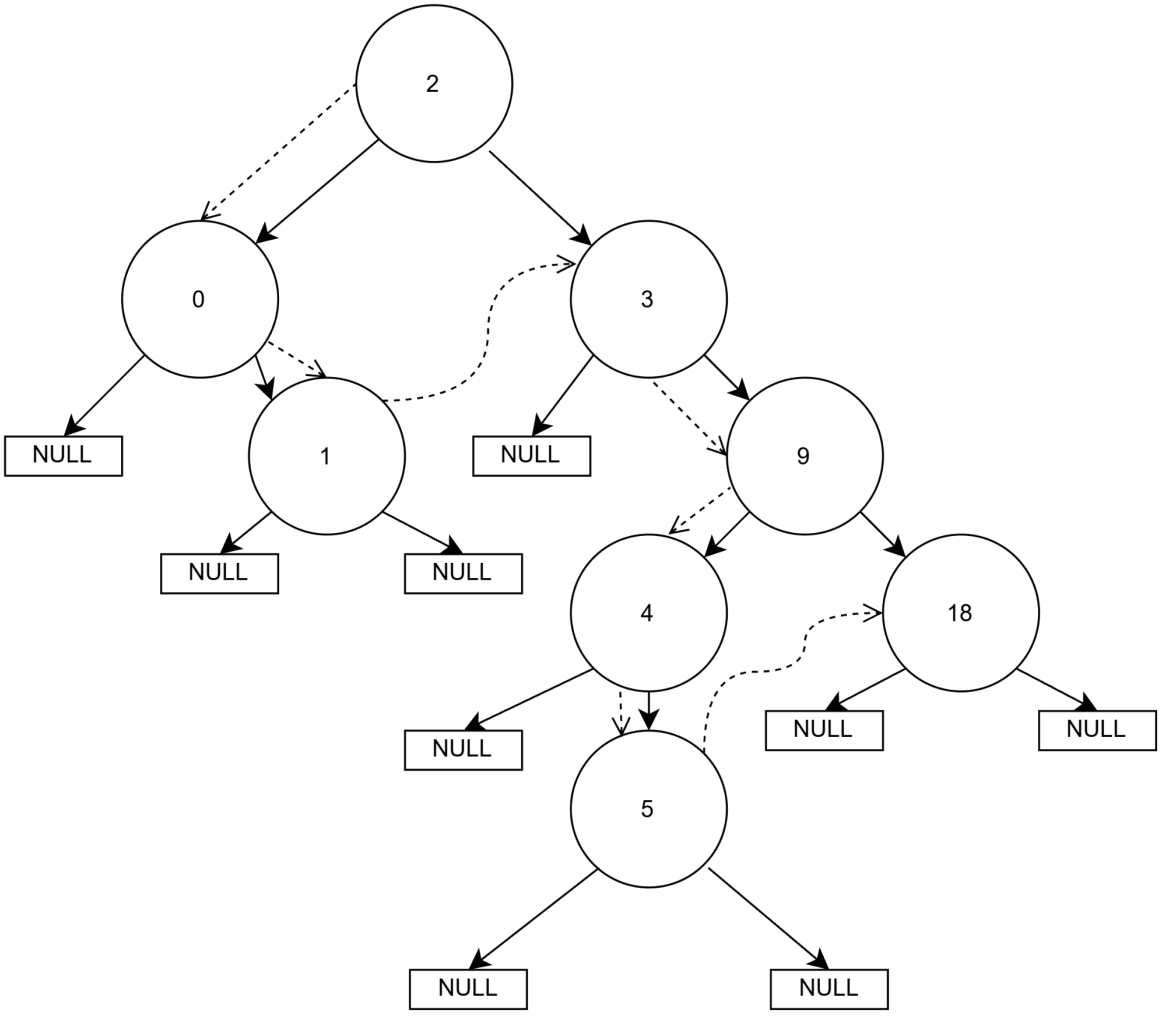
Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

Tree Diagram

In-order traversal



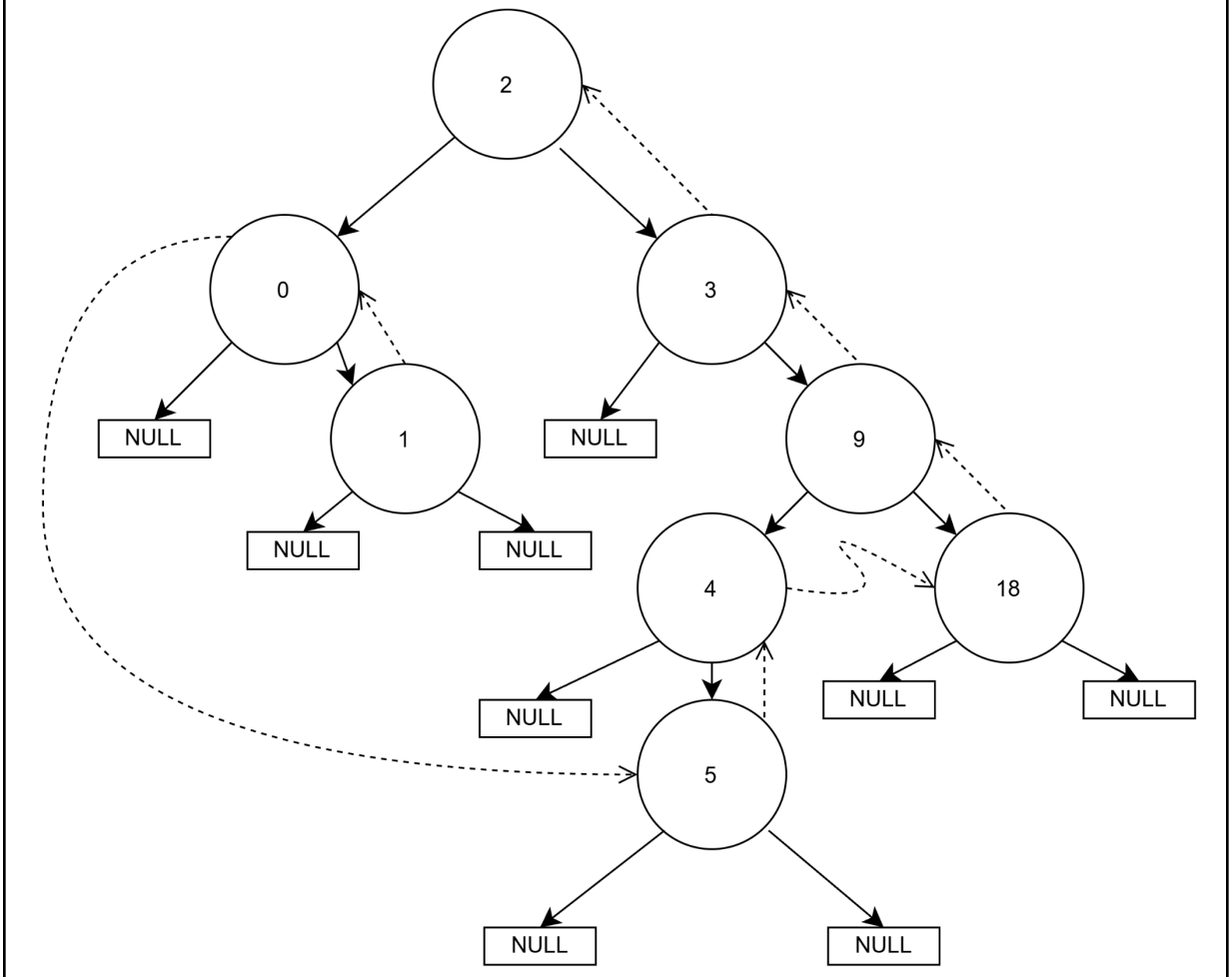
0 → 1 → 2 → 3 → 4 → 5 → 9 → 18

Pre-order traversel



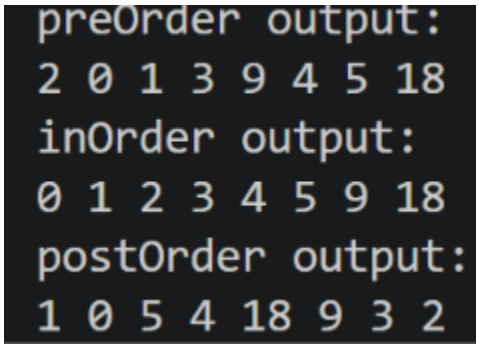
2 → 0 → 1 → 3 → 9 → 4 → 5 → 18

Post-order traversal



1 → 0 → 5 → 4 → 18 → 9 → 3 → 2

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

output


```
preOrder output:
2 0 1 3 9 4 5 18
inOrder output:
0 1 2 3 4 5 9 18
postOrder output:
1 0 5 4 18 9 3 2
```

answer


```
template<typename T>
void BinarySearchTree<T>::inOrder(){
    if(left != nullptr){
        left->inOrder();
    }
    std::cout << key << " ";

    if (right != nullptr){
        right->inOrder();
    }
}
```

observation
Both are correct, as in-order traversal of a binary tree strictly follows a left → root → right → pattern.

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

Function

```
50  template<typename T>
51  void BinarySearchTree<T>::preOrder(){
52      std::cout << key << " ";
53      if(left != nullptr){
54          left->preOrder();
55      }
56      if (right != nullptr){
57          right->preOrder();
58      }
59  }
```

Output

```
preOrder output:
2 0 1 3 9 4 5 18
```

Observation

Both are correct, as pre-order traversal of a binary tree strictly follows a root → left → right pattern.

Post-order Traversal

Function

```cpp
template<typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }

    if (right != nullptr){
        right->postOrder();
    }
    std::cout << key << " ";
}
```

Output

```
postOrder output:
1 0 5 4 18 9 3 2
```

Observation

Both are correct, as post-order traversal of a binary tree strictly follows a left–right–root pattern.

Conclusion:

In this activity, I learned how to implement general trees, binary trees, and binary search trees using C++. I also learned how pre-order, in-order, and post-order traversal work and how each method produces a different sequence of nodes. The procedure helped me understand how node height and depth are determined and how values are organized within a tree. The supplementary activity strengthened my understanding by allowing me to create a binary search tree and compare its different traversal outputs. Overall, I think I performed well, but I still need to improve my understanding of tree structures and traversal algorithms so I can solve similar problems more confidently and efficiently.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and ChatGPT**

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly was used for grammar correction, while ChatGPT was used to explain concepts from the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.